FROSTBIT：去中心化交易所技术白皮书

基于 Snowman 共识与 FROST 门限签名的跨链资产管理

Mao

2026 年 4 月 26 日

摘要

加密货币交易所长期面临两个结构性矛盾：一方面，用户期望获得中心化交易所般的低延迟体验；另一方面，频繁发生的交易所跑路、黑客盗币事件又不断提醒行业——将资产托付给任何单一实体，本质上是一种不可持续的信任假设。本文提出一种去中心化交易所（DEX）架构，试图在这两个看似矛盾的目标之间寻找可行的工程折中。

在共识层面，系统采用 Snowman/Snowball 家族协议，通过反复随机采样在数秒内完成区块最终化，同时以 $O(kn)$ 的通信开销避免了传统 BFT 协议的二次方瓶颈。在跨链资产管理方面，系统基于 FROST 门限签名协议与 ROAST 协调层，在矿工集体中分散密钥管理，支持 Bitcoin（BIP-340 Taproot）、Ethereum/BNB（alt_bn128）、Tron（secp256k1 ECDSA）等多条公链的去中心化托管与提现，从根本上消除了单一私钥带来的系统性风险。

关键词：去中心化交易所、Snowball 共识、FROST 门限签名、ROAST、跨链、DKG

目录

1 引言 **4**

1.1 背景与动机 4

1.2 核心挑战 4

1.3 本文贡献 4

1.4 系统总览 5

2 共识协议 **5**

2.1 为什么选择 Snowball 5

2.2 协议参数 6

2.3 投票处理 6

2.4 Window 优先级与安全性 7

2.5 VRF 确定性采样 7

2.6 最终化证据 7

2.7 同步机制 8

3 FROST 门限签名 **8**
3.1 从多签到门限签名 8
3.2 密码学基础 8
3.2.1 Schnorr 签名 8
3.2.2 BIP-340 (Bitcoin Taproot) 9
3.2.3 多曲线支持 9
3.3 分布式密钥生成 (DKG) 9
3.3.1 协议流程 10
3.4 FROST 签名协议 10
3.4.1 Round 1：Nonce 承诺 10
3.4.2 Round 2：部分签名与聚合 10
3.5 ROAST 协调层 11

4 跨链提现流程 **11**
4.1 端到端概览 11
4.2 Vault 与 Tweak 地址 12
4.3 UTXO 选择与隔离 12
4.4 VM 验证 12
4.5 EVM 与 Tron 链 13

5 见证者：充值确认与博弈论安全 **13**
5.1 为什么需要见证者 13
5.2 身份与质押三段式 14
5.3 充值请求的三段式状态机 14
5.4 Forward Beacon 抗 Grinding 抽签 14
5.5 投票与共识阈值 15
5.6 公示期与挑战机制 15
5.7 仲裁：独立抽签的二级法庭 16
5.8 结算：奖励、罚没、退款 17
5.9 安全性论证 17

6 Vault 密钥管理 **18**
6.1 Vault 生命周期 18

7 安全性分析 **18**
7.1 共识层安全性 18
7.2 签名层安全性 19
7.3 跨链层安全性 19

8 性能特性 **19**

9 相关工作 **19**

10 总结与展望 **20**

1 引言

1.1 背景与动机

自 2009 年 Bitcoin 诞生以来，加密货币生态经历了爆发式增长。截至撰写本文时，全球加密资产总市值已超过万亿美元量级，日均交易量可达数百亿美元。在这一庞大的经济活动中，**交易所**扮演着不可或缺的枢纽角色——它是绝大多数用户进入加密世界的第一个入口，也是价格发现、流动性聚合和资产兑换的核心基础设施。

然而，当前的交易所格局存在深层矛盾。中心化交易所（CEX）凭借其低延迟撮合引擎和流畅的用户体验占据了绝大部分市场份额，但它们本质上要求用户将资产托管给一个单点实体。历史反复证明了这种模式的脆弱性：2014 年 Mt. Gox 倒闭导致 85 万枚 BTC 丢失 [8]；2022 年 FTX 的崩溃蒸发了超过 80 亿美元的客户资产 [9]；更不用说 QuadrigaCX、Celsius 等一系列事件。这些并非孤立的" 黑天鹅"，而是中心化托管模式的**系统性风险**——只要存在单一控制点，就永远存在被攻击、被挪用、被冻结的可能。

去中心化交易所（DEX）试图从架构层面消除这一风险。以 Uniswap 为代表的链上 AMM 模式证明了去中心化交易的可行性，但也暴露了其固有局限：高 Gas 费用、MEV（最大可提取价值）攻击、以及无法高效支持限价单和传统订单簿模式。另一方面，以 dYdX 为代表的方案虽然采用了链下撮合 + 链上结算的混合架构，但仍然依赖特定的中心化组件或单一排序器。

本文提出的 FROSTBIT 系统采用一种不同的路径：**专用链共识 + 分布式密钥管理**。交易撮合和结算在专用的高性能区块链上完成，而跨链资产由矿工集体通过门限签名管理，任何单个矿工都无法独立转移用户资产。

1.2 核心挑战

设计这样一个系统需要解决两个关键技术问题：

1. **高效共识**——传统的 PBFT 类协议在大规模网络中面临 $O(n^2)$ 的消息复杂度瓶颈，而 Nakamoto 共识（PoW/PoS）的确认延迟通常在分钟级别。对于交易所场景，我们需要的是**数秒级确认**与**高吞吐量**的结合。

2. **去中心化跨链资产管理**——用户在 BTC、ETH 等外部公链上的资产需要以某种方式" 桥接" 到 FROSTBIT 链。传统的多签方案（m-of-n）虽然分散了风险，但签名体积随参与者数量线性增长，且无法与原生签名兼容。我们需要的是一种**门限签名方案**——多人合作生成、但外部验证等价于单人签名的密码学机制。

1.3 本文贡献

本白皮书详细描述 FROSTBIT 系统的核心技术架构，主要贡献包括：

- 一套基于 **Snowman/Snowball** 的共识协议实现，引入 Window 优先级和安全逃生阀机制，在保障安全性的同时提供活性保证（第 2 节）；

- 基于 **FROST + ROAST** 的跨链门限签名架构，支持 4 条主流公链的多曲线签名（第 3 节）；
- 完整的 **Vault 生命周期管理**，包括 DKG 密钥生成、提现签名、密钥轮换和资产迁移（第 4 节和第 6 节）；
- 基于**见证者投票 + Forward Beacon 抽签 + 挑战仲裁**的跨链充值确认机制，以博弈论激励替代外部信任假设（第 5 节）；
- 对系统安全性和性能特征的分析（第 7 节和第 8 节）。

1.4 系统总览

从宏观视角看，整个系统可以划分为两大平面：**共识平面**负责交易排序和区块最终化，**跨链平面**负责与外部公链的资产交互。两者通过 VM（虚拟机）层解耦——共识层不关心交易的具体语义，VM 层不关心区块如何达成共识。

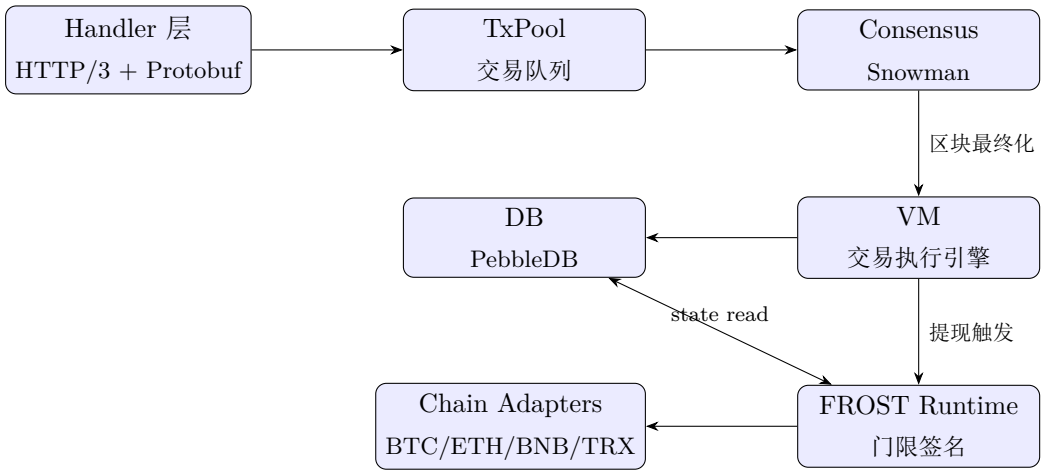

图 1: FROSTBIT 系统架构总览。交易从 Handler 层进入，经过 TxPool 缓冲和 Consensus 排序后，由 VM 执行。涉及跨链操作时，VM 触发 FROST Runtime 完成门限签名。

这种分层设计的好处是清晰的关注点分离：共识模块只负责" 就区块顺序达成一致"，VM 只负责" 给定一个区块，确定性地执行其中的交易"，FROST Runtime 只负责" 给定一个待签消息，协调矿工完成签名"。每一层都可以独立演进和测试。

2 共识协议

2.1 为什么选择 Snowball

区块链共识协议的设计空间中，存在几类经典范式。

Nakamoto 共识（如 Bitcoin 的 PoW）提供了优雅的概率性安全保证，但确认延迟在分钟到小时级别，吞吐量也受限于区块大小和出块间隔。**经典 BFT**（如 PBFT、Tendermint）可以实现即时最终性，但通信复杂度为 $O(n^2)$，在节点数量增长时迅速成为瓶颈。**基于 DAG**

的协议（如 Hashgraph、Narwhal-Tusk）通过有向无环图结构提升了吞吐量，但引入了较高的实现复杂度。

2019 年，Cornell 大学的 Emin Gün Sirer 团队提出了 **Avalanche 共识家族** [1]，开创了一条新的路径：通过**反复随机采样**实现共识。其核心直觉是——如果你反复问网络中的随机节点" 你偏好什么"，并跟随多数派调整自己的偏好，那么整个网络会迅速收敛到一致。

这一家族包含多个变体（Slush → Snowflake → Snowball → Avalanche → Snowman），每个变体在前者基础上增加了安全性或效率特性。本系统采用 **Snowman** 变体——它在 Snowball 投票机制之上增加了链式结构（每个区块引用父区块），适合我们需要的有序交易执行场景。

Snowball/Snowman 的关键优势在于：

- **通信效率**：每轮每节点仅采样 k 个节点，总通信量 $O(kn)$
- **快速收敛**：在诚实多数下，$O(\log n)$ 轮即可高概率收敛
- **无 Leader**：不需要 leader 选举，消除了单点故障
- **渐进安全**：安全性不依赖同步假设，在部分异步网络中仍然安全

2.2 协议参数

定义 2.1 (Snowball 参数). *协议由三个核心参数定义：*

- k：*每轮采样节点数（sample size）*
- α：*多数票阈值（quorum threshold），满足* $\lfloor k/2 \rfloor < \alpha \leq k$
- β：*最终化所需连续成功轮数（confidence threshold）*

直觉上，k 控制每轮采样的" 广度"——越大则每轮获得的信息越多，但通信开销也越大。α 控制" 多数判定" 的门槛——越大则越保守，需要更强的共识信号才会改变偏好。β 控制最终化的" 审慎度"——越大则需要更多轮连续确认，被逆转的概率指数级下降。

2.3 投票处理

每轮迭代中，节点随机采样 k 个节点并收集其偏好投票。投票结果的处理遵循以下核心逻辑：

- **成功路径**（得票 $\geq \alpha$）：如果赢家与当前偏好一致，置信度 $+1$；否则切换偏好并重置置信度为 1。
- **失败路径**（得票 $< \alpha$）：置信度归零。在连续失败超过逃生阈值 W_{esc} 后，允许切换偏好以保证活性。
- **最终化**：置信度达到 β 时，当前偏好被最终化。

这个设计中最关键的安全性不变量是：**偏好切换只在看到 α 多数票证据时发生**（失败路径中的逃生阀除外）。这确保了不会因为网络抖动或少数节点的偏好变化就轻易放弃已经积累的共识进展。

2.4 Window 优先级与安全性

在同一高度，可能存在多个候选区块——例如不同节点几乎同时提出提案。为了确保各节点在无通信的情况下对”哪个候选更好”达成一致判断，我们引入了 **Window 优先级机制**。

每个区块 ID 包含 `window` 编号，表示该区块的提出时间阶段。在需要打破平局时，系统总是偏好 Window 编号更小的区块：

$$\textsc{SelectBest}(C) = \arg\min_{c \in C} \big(\mathrm{window}(c),\ \mathrm{hash}(c)\big) \tag{1}$$

这一设计背后的安全性直觉是：Window 越小的区块存在时间越长，被更多节点接收和投票的概率越高，因此更可能代表全网的真实多数偏好。如果允许节点”免费”跳转到高 Window 区块，可能导致慢节点与快节点产生偏好分歧，最终引发分叉。

定理 2.1 (防双重最终化)**.** *设逃生阈值 $W_{esc} = 3\beta$，则在任何网络分区中，不会出现两个分区各自最终化不同区块。*

证明概要. 考虑网络分裂为快/慢两个分区。快分区最多在 β 轮内最终化某区块 B。慢分区中，偏好切换只能在两种情况下发生：(1) 看到 α 多数票（但分区内无法获得全网采样），或 (2) 连续失败 3β 轮后触发逃生阀。当慢分区终于准备切换时（至少 3β 轮后），快分区的最终化广播早已有充足时间到达。 □

2.5 VRF 确定性采样

为了使不同节点在相同条件下选择相同的采样集合（这对最终化证据的可验证性至关重要），系统使用确定性种子驱动的伪随机采样：

$$\mathrm{seed} = \mathrm{SHA256}(\mathrm{parentHash} \parallel \mathrm{height} \parallel \mathrm{window} \parallel \mathrm{nodeID}) \tag{2}$$

基于 seed 和批次号 seqID，通过 Fisher-Yates 洗牌从活跃节点列表中选取 k 个节点。这确保了给定相同的输入条件，任何节点都能独立复现相同的采样结果，为最终化证据的离线验证提供了基础。

2.6 最终化证据

区块被最终化时，系统收集连续 β 轮成功投票的详细记录，包括每轮的投票者身份和密码学签名：

$$\mathrm{Evidence} = \{(\mathrm{round}_i, \mathrm{seqID}_i, \{(\mathrm{nodeID}_j, \mathrm{vote}_j, \mathrm{sig}_j)\})\}_{i=1}^{\beta} \tag{3}$$

这些证据写入区块头，供后续加入的节点和轻客户端验证最终化的有效性。与传统 BFT 的 quorum certificate 类似，但这里的证据是**多轮**的——反映了 Snowball 渐进收敛的特性。

2.7 同步机制

对于新加入或落后的节点，系统提供两级同步策略：

增量同步——适用于轻微落后（差距在数百区块以内）的情况。节点定期向随机节点查询其最终化高度，当 $\geq \alpha$ 个节点报告的高度超过本地时，逐块拉取并验证缺失的区块。

深度同步（State Snapshot）——适用于严重落后的情况。直接下载目标高度的完整状态快照，跳过中间所有区块的重新执行。状态的完整性通过区块头中的 `StateRoot` 字段验证：

$$\text{verify} : \textsc{Fingerprint}(\text{snapshot}) \stackrel{?}{=} \text{block}[H+1].\text{StateRoot} \tag{4}$$

深度同步将一个可能需要数小时的逐块重放过程压缩到了分钟级别，这对于在生产环境中快速恢复故障节点至关重要。

3 FROST 门限签名

3.1 从多签到门限签名

在去中心化系统中管理外部公链的资产，最朴素的方案是**多签（Multisig）**：n 个参与者各自持有独立私钥，转移资产需要其中 m 个签名。Bitcoin 原生支持多签脚本，Ethereum 上也有 Gnosis Safe 等成熟实现。

然而，多签方案存在几个实质性缺陷：

- **签名体积线性增长**——m 个签名需要 m 倍存储空间，链上验证成本高
- **隐私泄露**——所有签名者的公钥和参与模式对外可见
- **不兼容原生验证**——部分链（如 EVM 系列）不原生支持多签脚本, 需要额外的合约层

门限签名（Threshold Signature）从根本上解决了这些问题。在门限签名方案中，n 个参与者**共享**一个私钥——没有任何单个参与者知道完整的私钥，但任意 t 个参与者合作可以生成一个标准签名。这个签名与单人签名**在密码学上不可区分**——验证者无需知道签名是由多人协作生成的。

本系统采用 **FROST**（Flexible Round-Optimized Schnorr Threshold Signatures）[2] 协议。FROST 的核心优势在于：(1) 只需两轮通信即可完成签名；(2) 可适配多种 Schnorr 变体（BIP-340、Ed25519 等）；(3) 签名产物与标准 Schnorr 签名完全兼容。

3.2 密码学基础

FROST 构建在 Schnorr 签名方案之上。我们简要回顾其核心构造。

3.2.1 Schnorr 签名

设 $\mathbb{G}$ 为素阶 n 的椭圆曲线群，G 为生成元。Schnorr 签名的三个步骤为：

1. **密钥生成**：随机选取 $x \xleftarrow{\$} \mathbb{Z}_n$，公钥 $P = xG$

2. **签名**：随机 nonce $k \xleftarrow{\$} \mathbb{Z}_n$，计算 $R = kG$，$e = H(R\|P\|m)$，$z = k + ex \bmod n$。签名为 $\sigma = (R, z)$。

3. **验证**：检查 $zG \stackrel{?}{=} R + eP$

Schnorr 签名的一个关键代数性质是**线性性**——这正是使其可以被”拆分”为多人协作的数学基础。如果 $x = \sum x_i$、$k = \sum k_i$，则 $z = \sum(k_i + e \cdot x_i)$，每个参与者只需计算自己的”部分签名”$z_i$，聚合后即可得到有效的完整签名。

3.2.2 BIP-340 (Bitcoin Taproot)

Bitcoin 的 Taproot 升级 [4] 引入了 BIP-340 Schnorr 签名标准，有几个与传统 Schnorr 不同的约束：

- 使用 `x-only` 公钥——仅 x 坐标（32 字节），隐式约定 y 为偶数
- $R.y$ 必须为偶数——否则取反 nonce k
- challenge 使用**域分离 (tagged) 哈希**：$e = H_{\text{tag}}(\texttt{"BIP0340/challenge"}, R_x\|P_x\|m)$

这些约束意味着在门限签名的实现中需要处理额外的 **parity 调整逻辑**——当 $R.y$ 或 $P.y$ 为奇数时，必须取反对应的 nonce 或密钥份额。这是 FROST 在 BTC 上落地的一个重要工程细节。

3.2.3 多曲线支持

不同公链使用不同的椭圆曲线和签名方案。系统通过一个统一的 `Group` 抽象接口封装曲线差异：

目标链	曲线	签名方案	公钥长度
Bitcoin	secp256k1	BIP-340 Schnorr	32 字节
Ethereum / BNB	alt_bn128	Schnorr	64 字节
Tron	secp256k1	ECDSA (GG20)	33 字节

表 1: 支持的签名算法与椭圆曲线

这种抽象使得新增一条链只需实现 `Group` 接口（提供基点乘法、点加法、序列化等操作），签名协议的上层逻辑无需修改。

3.3 分布式密钥生成 (DKG)

在使用 FROST 签名之前，首先需要在 n 个矿工之间分配密钥份额。这通过 **DKG（分布式密钥生成）**协议完成——本质上是 Feldman VSS（可验证秘密共享）[6] 的分布式版本。

协议的目标是：在没有任何可信第三方的情况下，使 n 个参与者各自获得一个密钥份额 s_i，满足 $s_i = f(i)$，其中 f 是一个度为 $t-1$ 的秘密多项式，且没有任何参与者知道 $f(0)$（即完整私钥）。

3.3.1 协议流程

1. **多项式生成**：每个参与者 j 独立生成随机 $t-1$ 阶多项式 $f_j(x) = a_{j,0} + a_{j,1}x + \cdots + a_{j,t-1}x^{t-1}$

2. **承诺阶段（Commit）**：广播多项式系数的椭圆曲线承诺 $C_{j,k} = a_{j,k} \cdot G$，$k = 0, \ldots, t-1$。承诺提交到链上，确保不可撤回和公开可验证。

3. **份额分发（Share）**：参与者 j 向每个参与者 i 发送密钥份额 $s_{j\to i} = f_j(i)$。

4. **份额验证**：参与者 i 验证收到的每个份额：

$$s_{j\to i} \cdot G \stackrel{?}{=} \sum_{k=0}^{t-1} i^k \cdot C_{j,k} \tag{5}$$

   若验证失败，可发起争议（dispute）。

5. **密钥聚合**：$s_i = \sum_j s_{j\to i}$（个人总份额），$Q = \sum_j C_{j,0}$（群公钥）

值得强调的是,在本系统中 DKG 的承诺和份额分发**都通过链上交易提交**（`FrostVaultDkgCommitTx` 和 `FrostVaultDkgShareTx`），而非通过点对点消息传递。这虽然增加了一些延迟，但换来了**完全可审计性**——任何人都可以从链上数据重放 DKG 过程，验证密钥份额的正确性。

3.4 FROST 签名协议

完成 DKG 后，选中的 t 个签名者即可协作生成签名。FROST 协议分为两轮。

3.4.1 Round 1：Nonce 承诺

每个签名者 i 生成一对随机 nonce (d_i, e_i) 和对应的椭圆曲线点承诺 $(D_i, E_i) = (d_i G, e_i G)$，并将承诺广播给协调者。

3.4.2 Round 2：部分签名与聚合

协调者收集到 $\geq t$ 个 nonce 承诺后，计算：

1. **绑定系数** $\rho_i = H\big(\texttt{"rho"} \| i \| m \| D_1 \| E_1 \| \cdots\big)$ ——将每个签名者的 nonce 与消息绑定，防止 nonce 重用攻击

2. **群承诺** $R = \sum_{i\in S}(D_i + \rho_i \cdot E_i)$

3. **挑战值** $e = H_{\text{tag}}(R_x \| Q_x \| m)$

4. **部分签名**——每个签名者 i 在本地计算：

$$z_i = d_i + \rho_i \cdot e_i + \lambda_i \cdot e \cdot s_i \pmod n \tag{6}$$

   其中 λ_i 为拉格朗日系数，用于从 t 个份额插值恢复完整秘密。

5. **聚合签名** $z = \sum_{i\in S} z_i \pmod n$，最终签名 $\sigma = (R_x, z)$

关键性质：最终的签名 σ 满足 $zG = R + eQ$，这与标准 Schnorr 签名完全相同。Bitcoin 网络、以太坊智能合约、BNB Smart Chain——所有验证者都**无法分辨**这个签名是由一个人还是一百个人生成的。

3.5 ROAST 协调层

FROST 的基本协议有一个隐含假设：所有被选中的 t 个签名者都在线且会诚实响应。在真实的异步网络中，这个假设过于理想化——节点可能因网络延迟超时、因故障宕机、甚至恶意拒绝响应。

ROAST（Robust Asynchronous Schnorr Threshold Signatures）[3] 正是为解决这一问题而设计的。它在 FROST 之上增加了一个协调层，核心机制包括：

1. **协调者选举与轮换**——通过确定性算法选举协调者：

$$\text{aggIdx} = \left\lfloor \frac{h_{\text{now}} - h_{\text{start}}}{R_{\text{blocks}}} \right\rfloor \bmod |\text{committee}| \tag{7}$$

   若当前协调者无法在 R_{blocks} 个区块内完成签名，系统自动切换到下一个候选协调者。

2. **确定性子集选择**——使用种子驱动的 Fisher-Yates 洗牌从 n 个成员中选取 t 个签名者。失败后使用不同的 retryCount 重新选择子集。

3. **超时与重试**——nonce 收集和签名份额收集各有独立的超时窗口。超时后丢弃当前子集，选择新子集重试。

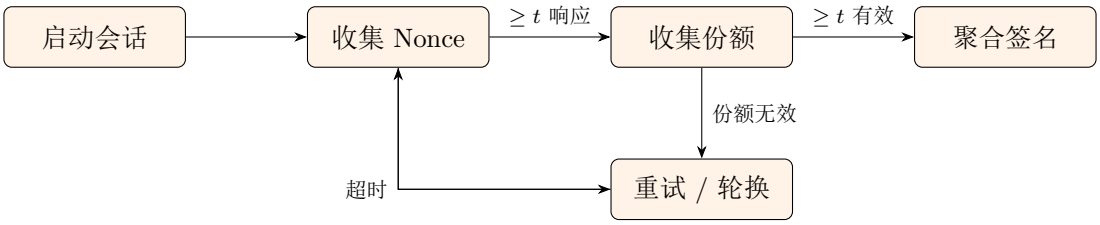

图 2: ROAST 协调流程。超时或份额验证失败时自动重试，协调者长时间失败后自动轮换。

ROAST 的核心保证是：只要网络中存在 $\geq t$ 个诚实且在线的签名者，签名**最终一定会完成**——无论其余节点处于何种状态（离线、缓慢、甚至恶意）。

4 跨链提现流程

4.1 端到端概览

有了共识保障交易排序、FROST 提供分布式签名能力之后，我们可以描述完整的跨链提现流程。以 BTC 提现为例，整个过程涉及以下步骤：

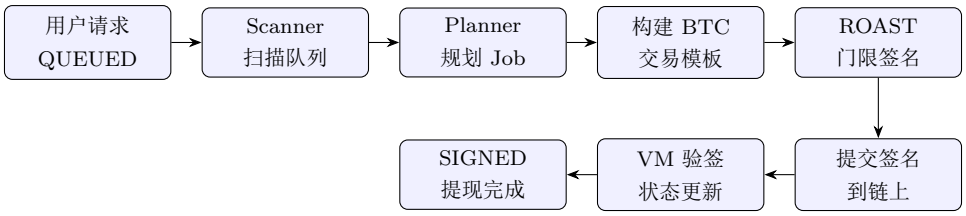

图 3: BTC 提现端到端流程

4.2 Vault 与 Tweak 地址

在 BTC 上，每个 Vault 拥有一个由 DKG 生成的群公钥 Q。为了让每个用户拥有独立的充值地址（而非所有用户共用一个地址），系统使用 **Tweak 地址方案**：

$$P_{\text{tweaked}} = Q + H(\text{userAddr}\|\text{vaultID}) \cdot G \tag{8}$$

由于 $H(\cdot) \cdot G$ 的离散对数未知，知道 Q 的私钥份额的签名者可以通过调整份额来签名（$s'_i = s_i + \lambda_i \cdot H(\cdot)$），但新的 tweaked 公钥与原始群公钥看起来完全无关。这同时实现了**隐私性**（用户地址互相独立）和**安全性**（一个地址的暴露不影响其他用户）。

4.3 UTXO 选择与隔离

BTC 的 UTXO 模型要求每笔提现交易指定输入 UTXO。为避免并发签名任务之间的 UTXO 冲突，系统采用**单一 UTXO 策略**：

- 每个签名 Job 只消耗 FIFO 队首的**一个** UTXO
- 找零输出回到同一 tweaked 地址
- 已选中的 UTXO 通过链上锁定标记防止双花
- 多个 withdraw 请求可合并到同一 Job（贪心装箱）

这种策略牺牲了一定的 UTXO 利用效率，换来的是显著简化的并发控制——不同 Job 之间天然不会产生 UTXO 冲突。

4.4 VM 验证

提现签名完成后，`FrostWithdrawSignedTx` 被提交到链上。VM 执行时进行以下验证：

1. 从 `template_data` 复算交易模板哈希，确保与提交的 `template_hash` 一致
2. 为每个 input 复算 Taproot sighash
3. 从链上状态读取每个 input UTXO 的 tweak 标量，计算 tweaked 公钥
4. 使用 tweaked 公钥和 BIP-340 验签逻辑验证每个 input 的签名
5. 检查 UTXO 未被双花（锁定标记）

6. 验证 FIFO 顺序（$\text{seq}_0 = \text{head}$，且连续递增）

所有验证通过后，相关 withdraw 请求的状态从 `QUEUED` 推进到 `SIGNED`。

4.5 EVM 与 Tron 链

对于 EVM 兼容链（Ethereum、BNB Smart Chain），提现通过部署在链上的 Schnorr 验证合约实现。合约接收 alt_bn128 曲线上的 Schnorr 签名，验证通过后执行 ERC-20 代币转账。

Tron 链复用 secp256k1 曲线上的 ECDSA 门限签名（GG20 路径），签名生成后由适配器封装为 TRC-20 转账交易广播至 Tron 网络。

所有链的差异由 `ChainAdapter` 接口封装：

接口方法	职责
`BuildWithdrawTemplate`	构建链专用交易模板
`TemplateHash`	规范化哈希（确保确定性）
`PackageSigned`	模板 + 签名 → 可广播交易
`VerifySignature`	VM 侧验证聚合签名

表 2: `ChainAdapter` 接口。新增链只需实现此接口并注册。

5 见证者：充值确认与博弈论安全

5.1 为什么需要见证者

提现路径上，矿工集合通过 FROST 门限签名直接生成原生链上的转账签名，密码学保证了”无人能盗取金库资产”。但充值方向并不对称——用户从 BTC、ETH 等外部公链向系统 Vault 地址转账，**这一事件本身发生在 FROSTBIT 链之外**，矿工无法通过共识协议直接观测到。系统必须有一种机制，将”该笔充值确实发生在外部链上”这一外部事实，**在不引入新的信任主体**的前提下转化为链上可验证的状态。

业内通用解法有两类：一是依赖 SPV 轻客户端或 zk 证明，将外部链区块头同步到本链上链上验证，这种方案密码学强度高但工程复杂度极大、对每条接入链都需要单独适配且对 EVM/Tron 等无简单 SPV 的链几乎不可行；二是依赖 Guardian/Oracle 集合或外部 Bridge 网络转发消息，但这相当于把信任锚从交易所搬到了一个新的中心化或半中心化集合，实质上回退到了本文一直在批判的单点托管假设。

本系统采用第三条路径：**由质押绑定的见证者集合进行投票确认，辅以挑战期与仲裁机制构成博弈论安全**。其核心思想是——任何人只要质押超过门槛即可成为候选见证者，被随机抽中后对充值事件投票表决；任何人都可以在公示期内对结果发起挑战；最终判决由独立抽取的仲裁者集合裁定，作恶方质押被罚没并按比例分给挑战者。密码学不能保证的事情，由经济激励兜底。

5.2 身份与质押三段式

成为见证者不需要任何许可,只需要发起一笔 `WitnessStakeTx` 锁定不少于 `MinStakeAmount` 的链上余额。退出同样必须经过明确的状态机，避免”作恶后立刻取款跑路”。完整生命周期分三段：

- **质押（ADD）**——`WitnessStakeTx`，`Op=ADD`:资金从 `Balance` 划入 `WitnessLockedBalance`，状态置为 `ACTIVE`，进入候选池。
- **退岗（REMOVE）**——`WitnessStakeTx`，`Op=REMOVE`：**仅切状态**为 `UNSTAKING`，记录 `UnstakeHeight`，但资金继续锁定。选择器立刻不再把该见证者纳入新请求，然而对已经接到的任务必须继续履约——这一设计避免见证者在察觉自己投了少数票之后立刻跑路逃避罚没。
- **领回（CLAIM）**——`WitnessClaimStakeTx`:在 `UnstakeHeight + UnstakeLockBlocks` 之后才能执行，资金从 `WitnessLockedBalance` 转回 `Balance`，状态置为 `EXITED`。

锁定期 `UnstakeLockBlocks` 既是给挑战者发现并发起挑战的窗口，也是给仲裁结果落盘的窗口。任何在锁定期内被判定为作恶的见证者，其 `WitnessLockedBalance` 在领回之前就会被按 `SlashRatio` 罚没——状态字段虽然是 `UNSTAKING`，但资金仍然在系统的扣押权范围内。

5.3 充值请求的三段式状态机

任何人观察到外部链上发生了一笔进入 Vault 地址的转账,都可以发起一笔 `WitnessRequestTx`，携带原生链 ID、原生交易哈希、收款地址、声明金额等字段。请求随后进入一个严格的三段式状态机：

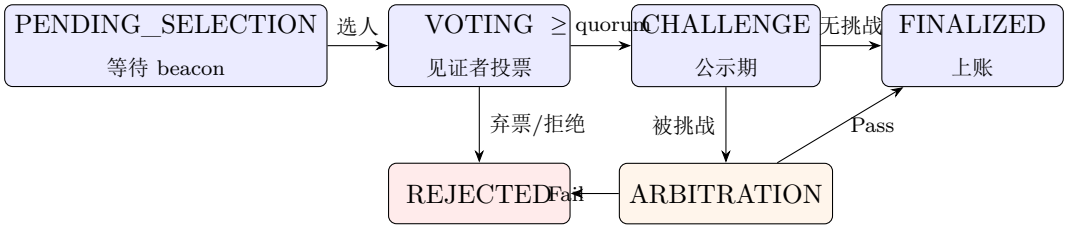

图 4: 充值请求的状态机

每个状态之间的转移都由区块高度驱动而非物理时间，保证所有节点在重放历史时得到完全一致的结果。`VotingPeriodBlocks`、`ChallengePeriodBlocks`、`ArbitrationPeriodBlocks` 三个参数分别控制各阶段的窗口长度。

5.4 Forward Beacon 抗 Grinding 抽签

随机抽取见证者集合是整个机制的关键防御点。如果由提交请求的人能够预测或操纵谁会被抽中，那么他可以反复构造请求直到串通过的合谋见证者凑够 quorum （这就是所谓的

grinding 攻击）。业内常见做法是用区块哈希作为随机源，但只要请求人能控制提交时机，他依然能在多个候选块之间选择对自己最有利的那个。

系统采用 **Forward Beacon**（前向信标）抽签：请求提交后**不立即选人**，只建立 `PENDING_SELECTION` 记录并写入 `selection_height = commit_height + SelectionBeaconDepth`（默认 K=6）。等本地 `currentHeight` $\geq$ `selection_height + BeaconMixBlocks - 1`（默认 mix=3，即 K+2 之后）时，VM 才读取未来 `BeaconMixBlocks` 块的链上写入的 `KeyBlockHashByHeight` 混合 `native_chain` ‖ `native_tx_hash` 生成 beacon：

$$\text{beacon} = H\big(\text{blkHash}_K \parallel \text{blkHash}_{K+1} \parallel \text{blkHash}_{K+2} \parallel \text{native_chain} \parallel \text{native_tx_hash}\big) \tag{9}$$

对请求方来说，他必须在提交时就承诺一个交易哈希，而决定 beacon 的区块哈希是 K 块之后才出现的——要操纵抽签结果，他得同时贿赂连续 K+2 块的出块者，而 grinding 的边际成本随 K 指数级上升。对矿工来说，每次想偏置 beacon 都要丢弃一个合法区块，经济成本由出块奖励直接限制。

抽签使用种子驱动的 Fisher-Yates 洗牌，从全网 `ACTIVE` 见证者池中无偏地抽取 `InitialWitnessCount` 名（默认 5 名）。`ApplyRechargeSelection` 是纯函数，同样的 (req, beacon, height) 输入永远产出相同的 (witnesses, VOTING request) 输出，**这是确定性共识与异步抽签共存的关键不变量**。

5.5 投票与共识阈值

被选中的见证者各自独立查询外部链全节点，核对原生交易哈希、收款地址、金额、确认数等关键字段，在 `VotingPeriodBlocks` 窗口内提交 `WitnessVoteTx`：赞成（PASS）、反对（FAIL）或弃票（ABSTAIN）。

投票汇总规则：

- **共识通过**：PASS 比例 $\geq$ `ConsensusThreshold`（默认 60%）且 ABSTAIN 比例 $<$ `AbstainThreshold`（默认 20%）
- **弃票超标**：ABSTAIN 比例 $\geq$ `AbstainThreshold` $\rightarrow$ 扩大见证者集合重选
- **扩容**：每轮按 `ExpandMultiplier` 倍扩展，最多若干轮，无法达成则进入 `REJECTED`

值得强调的是，弃票和反对在博弈论上是不同的——弃票意味着”我无法核实”（例如外部链 RPC 暂时失败），反对意味着”我已核实，事件不存在或不一致”。高弃票率触发扩容而不是直接拒绝，避免了少量节点的临时失联导致正常充值被误杀。

5.6 公示期与挑战机制

投票通过后请求进入 `CHALLENGE`（公示期）状态，在 `ChallengePeriodBlocks` 内任何人都可以发起 `WitnessChallengeTx`。关于挑战押金的定价，系统刻意做了一个非直觉的设计选择：

挑战押金与充值金额**脱钩**。

业内不少桥的做法是按充值金额线性抬高挑战门槛，理由是”金额越大攻击收益越大，应让挑战变贵”。本系统认为这一逻辑**方向恰好相反**：若按金额线性收押金，则攻击者只需虚构一笔 1 亿美元的假充值，就能让挑战者付不起押金而被迫放弃，最终见证者共识通过的任意金额都会顺利上账。系统安全性**依赖至少一个诚实挑战者能够在公示期内成功提交挑战**，所以挑战门槛必须按”仲裁成本 + 被挑战见证者可罚没的质押规模”定价，而不是按声明金额定价。

具体规则：

- 最小押金 `ChallengeStakeAmount`（足以覆盖一次仲裁的基础成本 `ChallengeArbitrationBaseCost`）
- 按可罚没池规模乘以 `ChallengeStakeRatioBps`（默认 10%）
- 上限由 `MaxChallengeStakeAmount` 兜底，0 代表不设限

挑战押金**挑战成功则退还本金 + 按贡献占比分得罚没池**，失败则全额销毁。这构成了挑战者的正向激励：只要有合理证据，期望收益为正。

5.7 仲裁：独立抽签的二级法庭

挑战触发后请求进入 `ARBITRATION` 状态。系统再次通过 Forward Beacon 抽取一组独立的仲裁者（同样从 `ACTIVE` 见证者池抽取，通过 beacon 中混合 `challenge_id` 与原投票池强制隔离）。仲裁者在 `ArbitrationPeriodBlocks` 内对原始结论投出 PASS（维持原判）或 FAIL（推翻原判）。

仲裁结算是整个机制中最复杂的一环，分两条路径触发：

- **同步路径**（投票达阈值）——`ArbitrationVoteTxHandler` 即时调用 `ProcessArbitrationVote`，达到 PASS/FAIL 判定阈值时立刻结算。
- **超时兜底路径**（沉默卡死）——VM 层 `applyArbitrationTimeouts` 在每个区块入口扫描超期的挑战记录，通过纯函数 `ComputeTimeoutFinalization` 强制按现有票数结算。

超时路径的存在是为了堵住”沉默杀死系统”这一活性漏洞——如果仲裁者集体不投票，没有兜底机制，挑战会永久挂起、请求永远无法 `FINALIZED` 也无法 `REJECTED`。为防止 1:0 的少数沉默票主宰判决，超时强制结算需满足 `canForceFinalize` 条件：已投人数 $\geq N/2$ 且 $\left|\text{Pass} - \text{Fail}\right| \times 5 \geq N$（约 20% 优势比）。不满足的请求继续按 `ExpandMultiplier` 扩容仲裁者池，阈值依次降低 80→70→60→50，候选耗尽后转入 `SHELVED`。进入 `SHELVED` 的请求最多停留 `MaxShelveWaitBlocks`（默认 200 块，生产建议 30 天）后自动 `REJECTED`，保守处置避免无限挂起。

5.8 结算：奖励、罚没、退款

无论从同步还是超时路径触发,最终结算共用同一套幂等逻辑 `applyChallengeResolutionLocked`:

- **诚实见证者**(投票方向与最终判定一致):按比例铸造 `RechargeFee` 分红,写入 `PendingReward`
- **作恶见证者 + 反向投票仲裁者**：按 `SlashRatio`（默认 100%）罚没 `StakeAmount`，状态置为 `SLASHED`，立即从候选池剔除
- **沉默仲裁者**(被选中但截止前未投票):**仅在超时结算路径**按 `SilenceSlashRatio`(默认 1%)轻罚,状态保留 `ACTIVE`。投票达阈值的同步路径不算沉默——此时仍在 `DeadlineHeight` 之前，他人可能尚未投递
- **挑战者**：成功 → 退本金 + 按贡献占比分罚没池；失败 → 押金 burn；`system` 自动挑战不参与结算

VM 落盘 `applySlashAndRefundToState` 一次性写入:SLASHED `WitnessInfo` + 扣 FB 锁定余额 + 加 challenger FB 余额。两次 `preExecuteBlock` 重跑产出完全一致的 WriteOp,**这是把异步博弈强行嵌入确定性共识的关键设计**:仲裁结果先写到 `sv.Set` 落盘的 `ChallengeRecord` 上，再由 `CommitTimeoutOutcome` 幂等同步回 Service 内存，节点内存抖动不影响落盘状态。

5.9 安全性论证

本机制成立的根本前提是：**在公示期内，至少存在一名愿意为正确性投入挑战押金的诚实参与者**。该假设比”超过 f 个矿工诚实”弱得多——任何旁观者只要观察到外链事实与请求声明不符，都有正向期望收益去发起挑战。

在该前提下：

- **抗合谋**——见证者抽签使用 Forward Beacon，请求方与矿工都无法预测或低成本影响抽签结果，串通成本随 `SelectionBeaconDepth` 指数级上升
- **抗沉默**——超时强制结算 + `SHELVED` 兜底确保任何请求最终都会得到判定，不会出现永久挂起；`canForceFinalize` 阻止少数票主宰
- **抗作恶**——见证者作恶将被 100% 罚没质押，挑战者按贡献占比分得罚没池，攻击经济不可行
- **抗 Tail-attack**——`UNSTAKING` 状态资金继续锁定 `UnstakeLockBlocks`，见证者无法在察觉作恶失败后立即跑路

值得指出的一点：见证者机制并不是要”代替”密码学保证，而是在密码学无能为力的边界（外部链事件观测）上，**用经济激励构造一个可证伪的诚实声明系统**。密码学保证”资产不会被偷”，见证者保证”虚假事件不会被采信”，两者共同构成了系统跨链层的完整安全闭环。

6 Vault 密钥管理

6.1 Vault 生命周期

密钥管理是跨链安全的根基。在本系统中，每条外部链上的资产由一个或多个 **Vault** 管理。每个 Vault 由 (`chain`, `vaultID`) 唯一标识，其密钥具有 `epoch` 版本号。Vault 的完整生命周期如下：

1. **创建**——通过链上交易 `FrostVaultCreateTx` 创建 Vault，触发首次 DKG
2. **DKG 执行**——委员会成员执行三阶段 DKG（Commit → Share → Validation），每个阶段有基于区块高度的截止时间
3. **正常运行**——Vault 接收用户充值，处理提现签名
4. **密钥轮换**——`FrostVaultTransitionTx` 触发新一轮 DKG，epoch 递增，新密钥生成后将旧 Vault 资产迁移到新公钥地址

密钥轮换是一个关键的安全实践。即使攻击者在某个时间窗口内成功获取了部分份额（但不足 t 个），定期轮换可以使这些份额失效，将攻击者的时间窗口限制在两次轮换之间。

图 5: DKG 三阶段流程。每个阶段通过链上交易提交，区块高度作为天然的时钟驱动 deadline。

7 安全性分析

一个完整的安全性分析需要考虑三个维度：共识层的安全性保证了交易不可篡改和回滚，签名层的安全性保证了跨链资产不可被盗取，跨链层的安全性保证了充值和提现的正确执行。

7.1 共识层安全性

- **活性**——只要诚实节点占多数 ($> n/2$)，Snowball 以压倒性概率在 $O(\log n)$ 轮内收敛到一致偏好。即使出现短暂的网络分区，逃生阀机制确保分区恢复后系统可以继续推进。
- **安全性**——Window 优先级 + 逃生阀 ($W_{\text{esc}} = 3\beta$) 的组合确保在任何分区场景下不会出现双重最终化。最终化证据（签名 Chits）提供了可验证的共识证明。
- **效率**——每轮通信量 $O(k)$（每节点），远优于 PBFT 的 $O(n^2)$。在典型参数设置下（$k = 20, \alpha = 15, \beta = 15$），15 轮即可最终化，总延迟约数秒。

7.2 签名层安全性

- **不可伪造性**——在离散对数假设（DLP）下，任何 $< t$ 个参与者的联合无法伪造有效签名。这由 FROST 协议的可证明安全性保障 [2]。
- **密钥安全**——DKG 确保完整私钥从未在任何单点出现。即使攻击者攻破了 $t-1$ 个节点，仍无法恢复完整私钥或伪造签名。
- **ROAST 容错**——只要 $\geq t$ 个节点诚实且在线，签名一定能完成。剩余 $n-t$ 个节点可以任意行为（离线、恶意、拜占庭）。

7.3 跨链层安全性

- **模板绑定**——`templateHash` 在链上由 VM 复算验证，确保签名对应的交易内容未被篡改。
- **UTXO 双花防护**——链上锁定标记确保已选中的 UTXO 不会被另一个 Job 使用。FIFO 严格顺序处理避免了乱序引发的状态不一致。
- **充值验证**——充值通过见证者投票 + 公示期 + 挑战/仲裁机制确认（详见第 5 节），博弈论激励确保见证者诚实报告。

8 性能特性

指标	特征
区块确认延迟	数秒级（β 轮采样，典型 $\approx$ 5s）
每轮通信量	$O(k)$ per node
签名门限	t-of-n（默认 $t = \lceil 2n/3 \rceil + 1$）
支持公链	BTC, ETH, BNB, TRX
DKG 轮次	3 阶段，各有区块高度 deadline
提现处理	批量 Job + 并发 ROAST 会话

表 3: 关键性能指标

9 相关工作

共识协议。本系统的 Snowman 共识源自 Avalanche 家族 [1]，与 Tendermint[10] 等经典 BFT 协议相比，优势在于无 Leader、通信效率高、在部分异步下仍安全。与 HotStuff[11] 的流水线 BFT 相比，Snowball 不需要 view change 协议，实现复杂度更低。

门限签名。 FROST[2] 在 Pedersen DKG[7] 的基础上增加了 Schnorr 签名的门限化。ROAST[3] 进一步解决了异步环境下的鲁棒性问题。与 MPC-based 方案（如 GG18/GG20 用于 ECDSA）相比，Schnorr-based 方案通信轮次更少、计算开销更低。

跨链桥。传统的跨链桥（如 Wormhole、LayerZero）通常依赖 Guardian 集合或 Oracle 网络进行消息中继。本系统直接在共识矿工集合中进行门限签名，避免了引入额外的信任假设层。

10 总结与展望

本文描述了完全去中心化交易所的核心技术架构。系统通过 Snowman 共识实现了高效的区块排序和最终化，通过 FROST + ROAST 实现了安全的跨链资产管理，两者的结合使得用户可以在**无需信任任何单一实体**的前提下进行高效的加密资产交易。

参考文献

[1] Team Rocket, Maofan Yin, Kevin Sekniqi, Robbert van Renesse, Emin Gün Sirer. *Scalable and Probabilistic Leaderless BFT Consensus through Metastability.* arXiv:1906.08936, 2019.

[2] Chelsea Komlo, Ian Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures.* SAC 2020, LNCS 12804, pp. 34–65.

[3] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, Dominique Schröder. *ROAST: Robust Asynchronous Schnorr Threshold Signatures.* ACM CCS 2022.

[4] Pieter Wuille, Jonas Nick, Tim Ruffing. *BIP 340: Schnorr Signatures for secp256k1.* Bitcoin Improvement Proposal, 2020.

[5] Adi Shamir. *How to Share a Secret.* Communications of the ACM, 22(11):612–613, 1979.

[6] Paul Feldman. *A Practical Scheme for Non-interactive Verifiable Secret Sharing.* FOCS 1987, pp. 427–437.

[7] Torben Pryds Pedersen. *Non-interactive and Information-theoretic Secure Verifiable Secret Sharing.* CRYPTO 1991, pp. 129–140.

[8] Kim Nilsson. *The Missing MtGox Bitcoins.* WizSecurity Technical Report, 2015.

[9] John J. Ray III. *Declaration in Support of Chapter 11 Petitions.* U.S. Bankruptcy Court, District of Delaware, 2022.

[10] Ethan Buchman. *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains.* M.Sc. Thesis, University of Guelph, 2016.

[11] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, Ittai Abraham. *HotStuff: BFT Consensus with Linearity and Responsiveness.* PODC 2019.